Assignment 1: The World of Zuul

The Game

Your task is to invent and implement an adventure game. Along with this document, you have been given a simple framework (*zuul-better*) that lets you walk through a few rooms. You can use this as a starting point.

1 Read the Code

Reading code is an important skill that you need to practice. Your first task is to read some of the existing code and try to understand what it does. By the end of the assignment, you will need to understand most of it.

2 Make small extensions

As a little exercise to get warmed up, make some changes to the code. For example:

- change the name of a location to something different.
- change the exits – pick a room that currently is to the west of another room and put it to the north
- add a room (or two, or three, ...)

These and similar exercises should get you familiar with the game.

3 Design Your Game

First, you should decide what the goal of your game is. It should be something along the lines of: You have to find some items and take them to a certain room (or a certain person?). Then you can get another item. If you take that to another room, you win.

For example:

> You are at Bush House, King's College London. You have to find out where your programming lab is. To find this, you have to find the department office and ask. At the end, you need to find the exam room. If you get there on time, and you have found your textbook somewhere along the way, and you have also been to the lab, then you win. And if you've been to the Waterfront bar to have a drink more than five times during the game, your exam mark halves.

Or:

> You are lost in a dungeon. You meet a dwarf. If you find something to eat that you can give to the dwarf, then the dwarf tells you where to find a magic wand. If you use the magic wand in the big cave, the exit opens, you get out and win.

It can be anything, really. Think about the scenery you want to use (a dungeon, a city, a building, etc) and decide what your locations (rooms) are. Make it interesting, but don't make it too complicated. (I would suggest no more than 12 rooms.)

Put objects in the scenery, maybe people, monsters, etc. Decide what task the player has to master.

4 Implement the Game

Decide what classes you need to implement the game, then implement and test them.

5 Levels

The base functionality that you have to implement is:

- The game has at least six locations/rooms.
- There are items in some rooms. Every room can hold any number of items. Some items can be picked up by the player, others can't.
- The player can carry some items with him. Every item has a weight. The player can carry items only up to a certain total weight.
- The player can win. There has to be some situation that is recognised as the end of the game where the player is informed that he/she has won.
- Implement a command "back" that takes you back to the last room you've been in. The "back" command should keep track of every move made, allowing the player to eventually return to its starting room.

- Add at least four new commands (in addition to those that were present in the code you got from us).

Challenge tasks:

- Add characters to your game. Characters are people or animals or monsters – anything that moves, really. Characters are also in rooms (like the player and the items). Unlike items, characters can move around by themselves.
- Extend the parser to recognise three-word commands. You could, for example, have a command

```
give bread dwarf
```

  to give some bread (which you are carrying) to the dwarf.
- Add a magic transporter room – every time you enter it you are transported to a random room in your game.

A good implementation of the base tasks will get you an average mark. To do better than that you need to implement one or more challenge tasks.

6 Report

You must also write a report (up to five pages) describing your game. The report should contain the following:

- The name and a short description of your game.
- The description should include at least a user level description (what does the game do?) and a brief implementation description (what are the important features?).
- A bullet point list of each base task you completed and how you completed it.
- A bullet point list of each challenge task you completed and how you completed it.
- For each of the following code quality considerations, give and explain an example in your project where you considered it: coupling, cohesion, responsibility-driven design, maintainability.
- A walk-through of your game, consisting of the commands that need to be entered to complete/win the game.

7 Submission and Assessment

You must submit your work via the "assignment 1 submission" link in the assignment 1 section on the PPA KEATS page, before **Thursday, 4 December 2025, 4pm.** You'll submit a zip file containing the following **three** parts:

1. **A Jar file** of your BlueJ project.

   – You can create a Jar from within BlueJ by going to Project, and then "Create Jar File...".

   – You do not need to change any of the default options, and so you should just click the "Continue" button.

   – The Jar file must contain your source code, i.e., the *.java files, and it must run on BlueJ.

2. **A report** (pdf file).

3. **All of your Java files** (although the Jar file contains the Java files, these extra Java files in the submission package make it easier for markers to add more targeted and specific feedback on specific places in your code in Gradescope).

Follow all instructions in the 'Student Submission Guide'. If you have any trouble submitting your work, email Jie Zhang (jie.zhang@kcl.ac.uk) as soon as possible. We recommend submitting your work well before the deadline to avoid last-minute technical issues.

Your submission will be assessed for

- completeness
- correctness
- appropriate use of language constructs
- style (commenting, indentation, etc.)
- design (consideration given to cohesion, coupling, maintainability)
- difficulty (extra marks for challenging tasks)

8 Submission Deadline

This assignment (code and report) is due at

Thursday, 4 December 2025, 4pm

Late submissions within 24 hours after the deadline will be capped at 40%. Submissions later than 24 hours will receive a mark of 0.